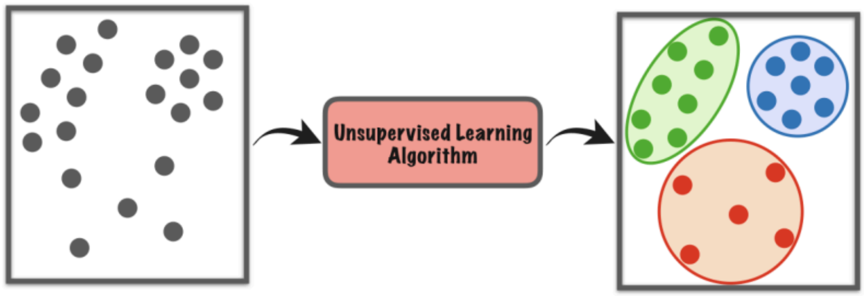
UNSUPERVISED MACHINE LEARNING

MODULE 1:

INTRODUCTION TO UNSUPERVISED MACHINE LEARNING AND K MEANS

TABLE OF CONTENTS

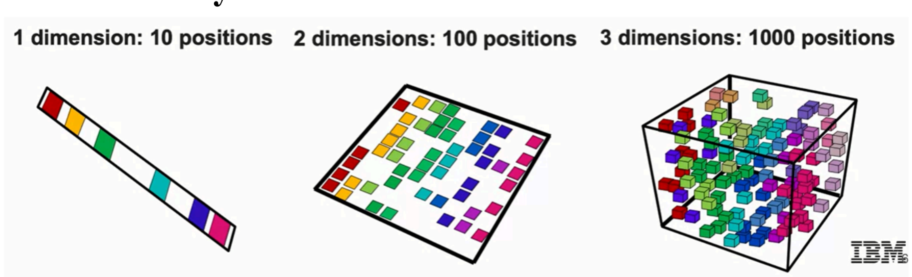
Unsupervised Learning Introduction



Main ideas:

- Unsupervised learning is used when we **don’t have labeled data** or target variables.
- Goal: **find hidden structures** or **group data** into smaller, meaningful subsets.
- Two main types:
    - **Clustering**: groups unlabeled data (e.g., customer segmentation).
    → Algorithms: K-Means, Hierarchical Agglomerative Clustering, DBSCAN, Mean Shift.
    - **Dimensionality Reduction**: simplifies data while retaining key information.
    → Algorithms: Principal Component Analysis (PCA), Non-negative Matrix Factorization (NMF).

Curse of Dimensionality:



- More features ≠ always better performance.
- Too many dimensions cause:
    - Spurious correlations
    - More noise than signal

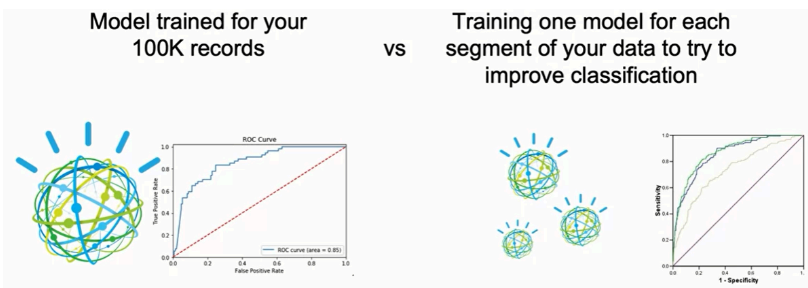
- Exponential increase in data needed
- Slower computation & more outliers

- Solution: reduce dimensionality (e.g., PCA)

Real-world Applications

Clustering use cases:

- **Classification:** leverage pattern in unlabeled data (e.g., spam filter, group of product reviews)
- **Anomaly Detection:** detect fraud patterns or outliers (e.g., suspicious transactions).
- **Customer Segmentation:** group customers by demographics, recency, frequency, or engagement.
- **Improve Supervised Learning:** train separate models per cluster (e.g., multiple logistic regressions).



Dimensionality Reduction use cases:

- **Image Compression:** reduce high-resolution images while preserving structure.
- **Image Tracking:** remove noise, increase speed in video/object tracking

Clustering Overview

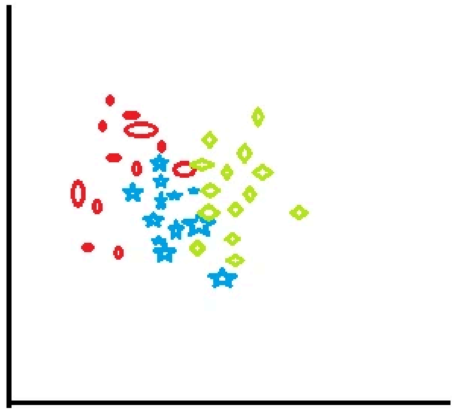


fig 1: before applying k-means clustering

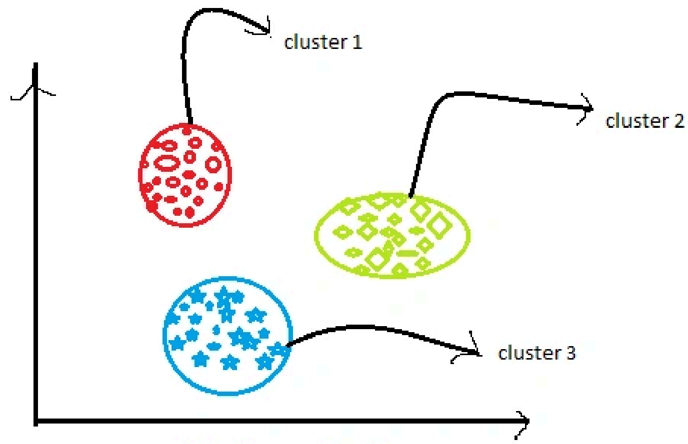
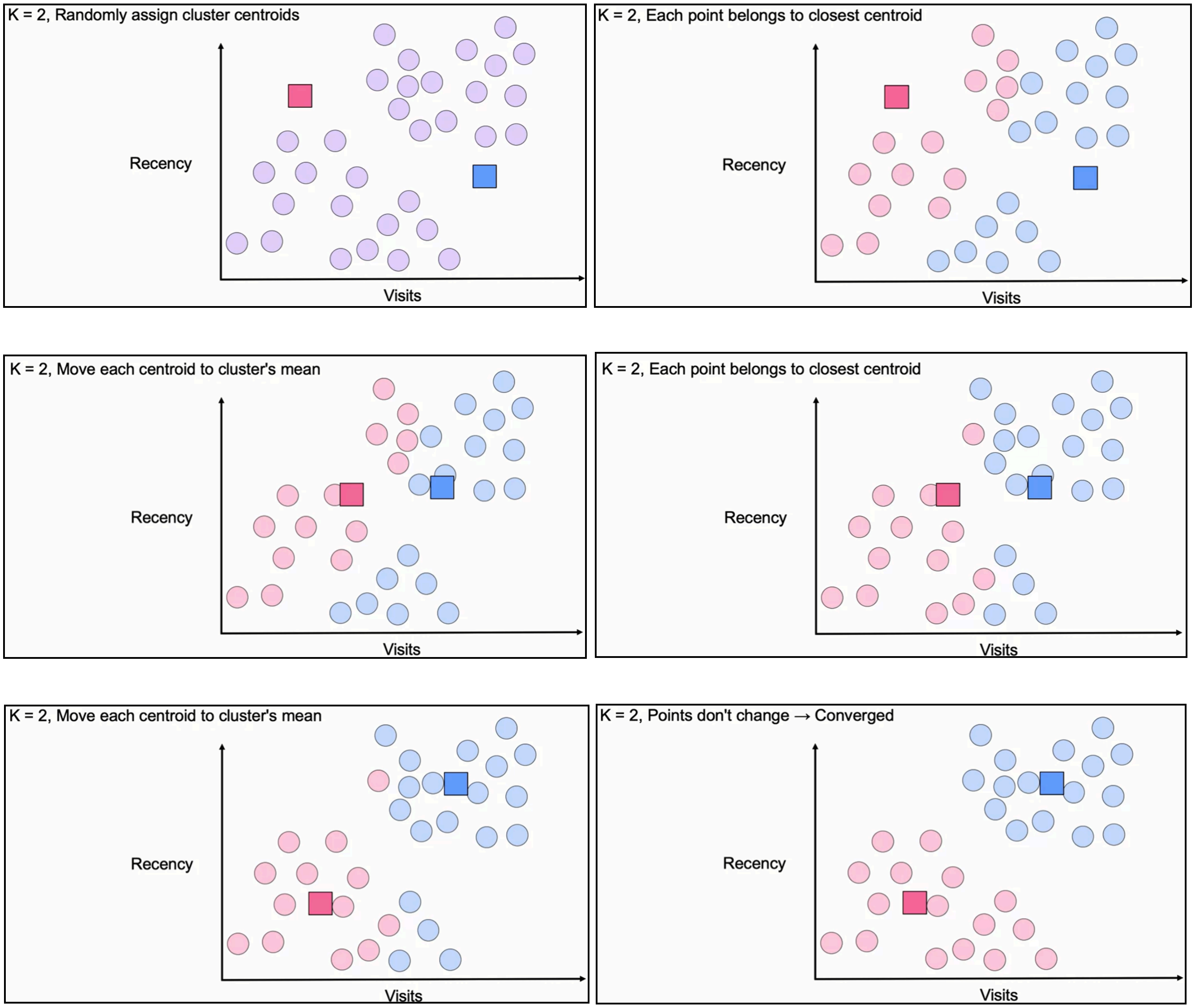


fig 2: After applying K-means clustering

- **Concept:**
  - Visual segmentation of data into **2, 3, or more clusters**.
  - The number of clusters depends on business needs or data patterns.
- Clustering divides data based on similarity; selecting the right number of clusters is crucial.

K-Means Algorithm

- **Working principle:**
  - Randomly initialize **K centroids**.
  - Assign each point to the nearest centroid (based on distance).
  - Recalculate centroids as the **mean of assigned points**.
  - Repeat until **no point changes cluster** → convergence.



- **Visualization idea:** After multiple iterations, centroids stop moving → clusters stabilize.
- **Advantages:** simple and efficient.
- **Disadvantages:** sensitive to initial centroids → may yield different results.

```
from sklearn.cluster import KMeans

# Initialize model
kmeans = KMeans(n_clusters=3, init='k-means++')
# Fit model
kmeans.fit(X)
```

```
# Predict clusters
labels = kmeans.predict(X)
```

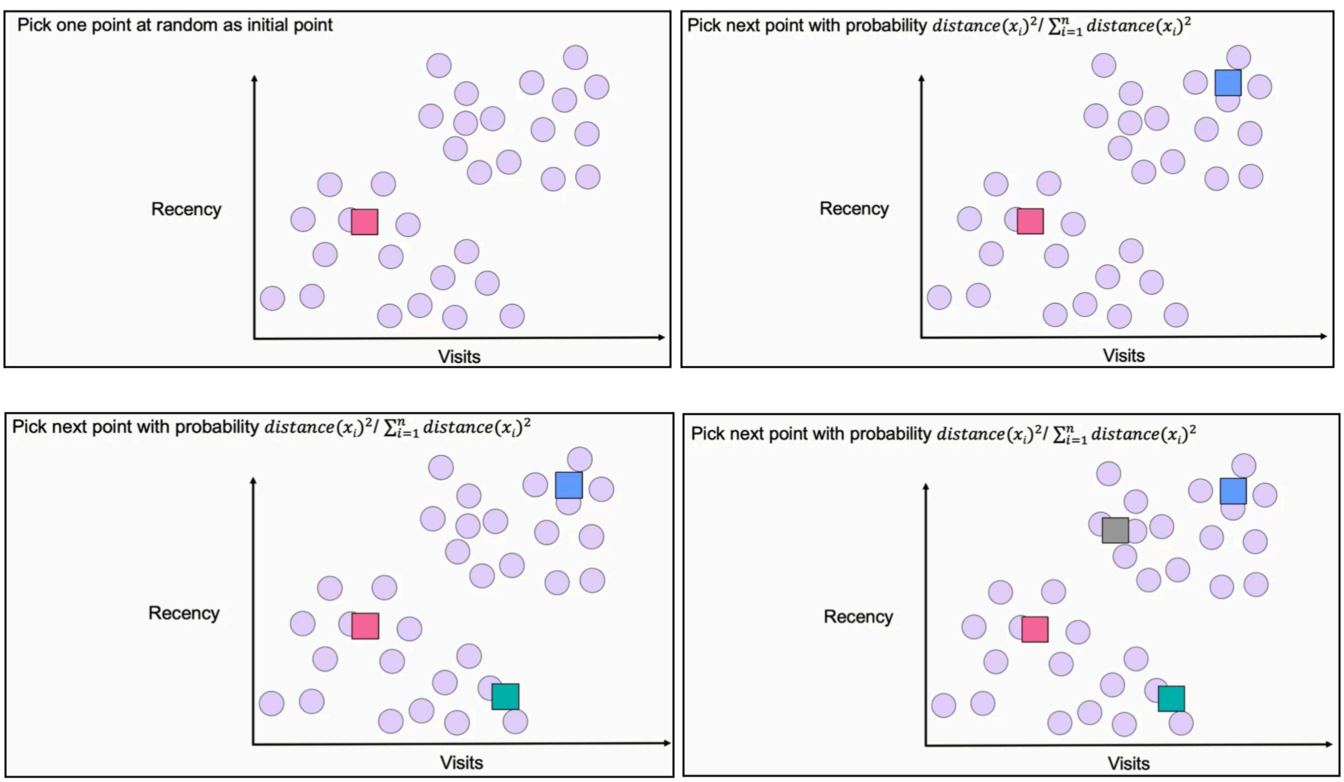
K-Means++ Initialization

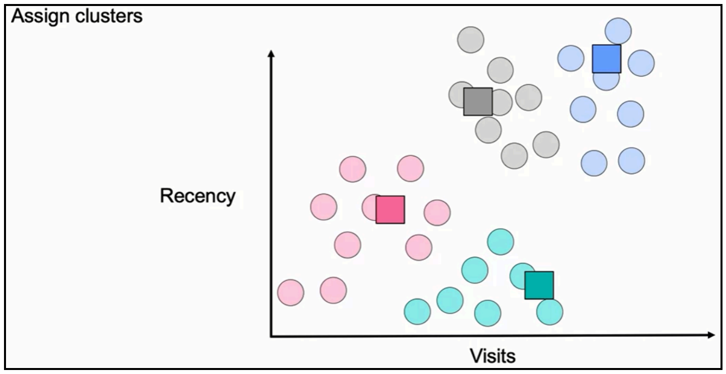
Problem:

- Standard K-Means can converge to suboptimal results due to poor initialization.

Solution – K-Means++:

- First centroid chosen randomly.
- Subsequent centroids chosen probabilistically:
  - Weight = (distance² from nearest existing centroid) / (sum of all distances²)
- Ensures centroids are **spread out** to avoid local minima.





Choosing the Number of Clusters (K)

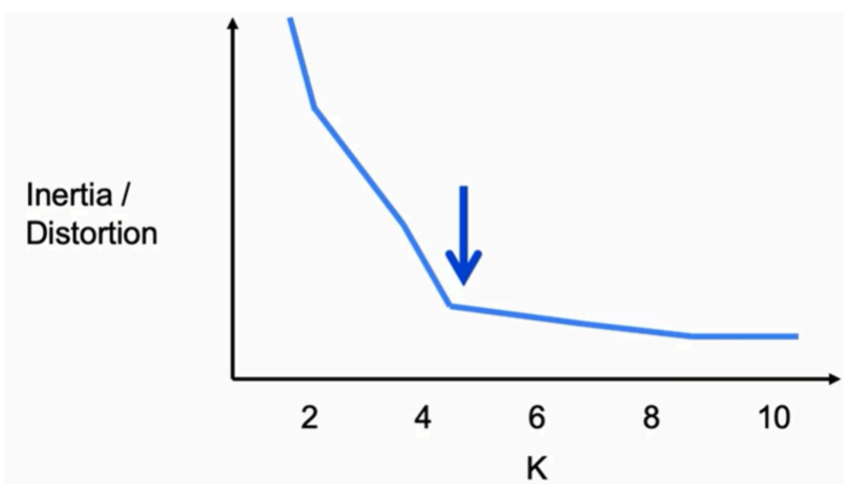
When K is unknown:

- Metrics to evaluate clustering quality:
  - **Inertia:** sum of squared distances from points to their centroid.
    → Smaller = tighter clusters; sensitive to cluster size.
  - **Distortion:** average squared distance from points to centroid.
    → Smaller = tighter clusters; less sensitive to cluster size.

Guidelines:

- Use **Inertia** when equal-sized clusters matter.
- Use **Distortion** when cluster similarity matters.

Finding the Right K – The Elbow Method



Concept:

- Plot **Inertia (or Distortion)** vs **number of clusters (K)**.
- Look for an "**elbow point**" — where the rate of improvement sharply decreases.
  → Indicates the optimal K (balance between accuracy and simplicity).

```
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

inertia = []
K_range = range(1, 10)

for k in K_range:
    model = KMeans(n_clusters=k, init='k-means++')
    model.fit(X)
    inertia.append(model.inertia_)

plt.plot(K_range, inertia)
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Inertia')
plt.title('Elbow Method')
plt.show()
```

Key Takeaways

- **Unsupervised learning** uncovers patterns in unlabeled data.
- **Clustering** organizes data into groups; **Dimensionality Reduction** simplifies datasets.
- **K-Means** is the foundational clustering algorithm, with **K-Means++** improving its reliability.
- **Model evaluation metrics:** Inertia and Distortion.
- **Elbow Method** determines the best number of clusters.
- Applications span **fraud detection**, **customer segmentation**, **image processing**, and **model optimization**.